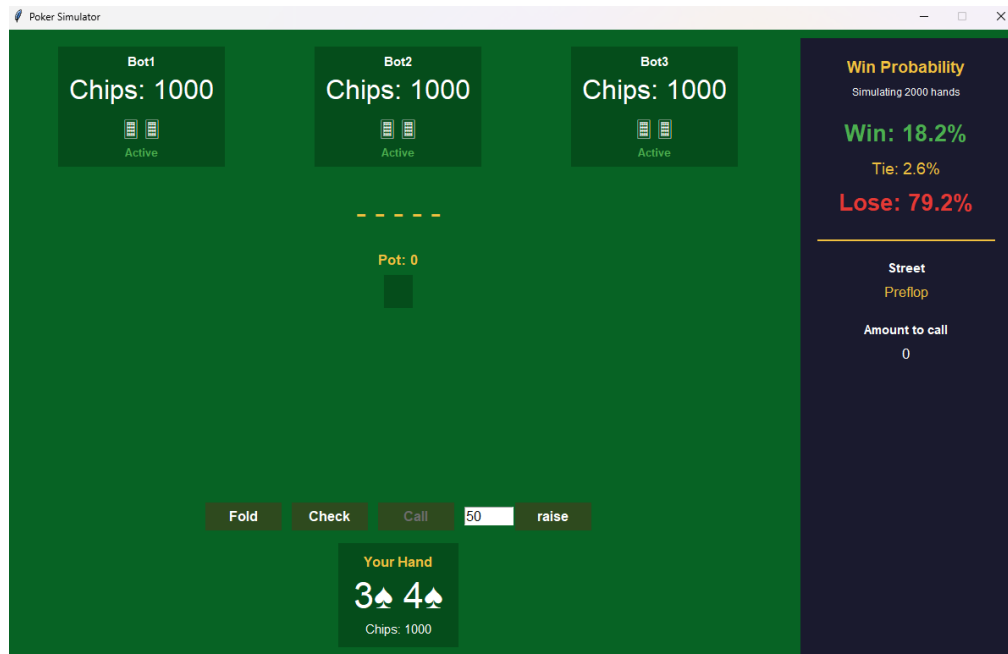


# Python Texas Hold'em Poker Simulator



## 1. Project Overview

A fully playable Texas Hold'em poker simulator built entirely in Python. The player competes against three AI-powered bots across a complete game loop, including preflop, flop, turn and river betting rounds. The main feature is a live probability analysis panel on the right-hand side of the window, which updates after every street to show the player's real-time estimated chance of winning, tying, or losing the hand. This allows users to make moves based on actual probabilities.

The project was built independently, with the goals of deepening practical Python skills, including my first GUI, exploring probability and simulation techniques, and integrating an AI API into a working application.

**DISCLAIMER: For AI bot functionality, users must generate their own Gemini API key. Available at <https://aistudio.google.com>**

## 2. Key Features

### 2.1 Live Win Probability Panel

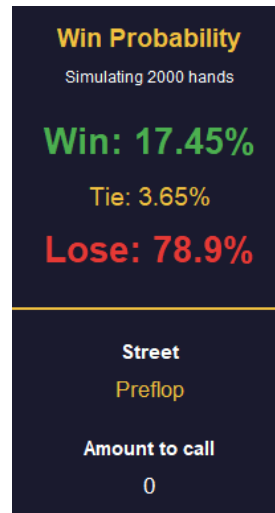
The main reason for this project was the real-time probability panel. After each card is dealt the panel updates to show the player's current estimated win, tie and loss percentage against the remaining active opponents.

This is calculated using a Monte Carlo simulation. Rather than mathematically enumerating every possible remaining hand combination, the simulator instead:

- Randomly assigns hole cards to each opponent from the remaining deck

- Randomly completes the board with the remaining community cards
- Evaluates who wins that scenario using the treys hand evaluation library
- Repeats this process 2,000 times per calculation
- Reports the percentage of simulations won, tied and lost as the estimated probability

Running 2,000 simulations consistently converges to an accurate probability estimate in well under a second, making it fast enough to update after every player's turn.



## 2.2 Texas Hold'em Game Loop

The simulator implements a complete Texas Hold'em hand, including:

- Dealing two hole cards to each player from a shuffled 52-card deck
- Preflop, flop, turn and river betting rounds with correct order of play
- No-limit betting — the player can raise by any amount up to their chip stack
- Fold, check, call and raise actions with input validation
- Correct handling of raises mid-round so all remaining players must respond
- Pot management, chip tracking and automatic showdown resolution
- Automatic start of the next hand after a 4-second showdown display

## 2.3 Google Gemini AI Bots

I began using a coded bot logic, which no matter how hard I tried, I could not replicate the randomness of a human poker player. So I implemented a system to use an API to control the bots individually, allowing for a much more realistic game.

Each of the three bots is powered by Google Gemini, a large language model API. Rather than following a fixed decision tree, each bot sends a structured prompt to the Gemini API describing its hole cards, the community cards, the pot size, the amount to call, its estimated win probability and its personality. Gemini returns a decision and a short one-sentence explanation of its reasoning, which is displayed in the GUI during play.

The three bots have distinct personalities assigned in the prompt, which can be changed to fit the player's desired opponents:

- Bot 1 (Aggressive) — bluffs often, raises frequently, and puts pressure on opponents

- Bot 2 (Passive) — prefers to call rather than raise, rarely bluffs, and folds when unsure
- Bot 3 (Balanced) — makes decisions based on pot odds and hand strength

Because Gemini interprets the personality description in context each time, bot behaviour varies naturally in ways that a fixed rule-based system could not replicate.

```
Calling Gemini for Bot1 (attempt 1)...
Gemini responded: ACTION: fold
AMOUNT: 0
REASON: My hand is too weak with a very low win probability against multiple
Calling Gemini for Bot2 (attempt 1)...
Gemini responded: ACTION: fold
AMOUNT: 0
REASON: My Q3s hand is too weak to open with for a balanced player against mu
Calling Gemini for Bot3 (attempt 1)...
Gemini responded: ACTION: check
AMOUNT: 0
REASON: My hand is very weak, but I can see the flop for free to re-evaluate
```

## 2.4 API Key Setup and Fallback Logic

The Gemini API requires a personal API key obtained for free from Google AI Studio ([aistudio.google.com](https://aistudio.google.com)). The key is stored securely in a .env file in the project root, which is excluded from version control via .gitignore. The python-dotenv package loads the key at runtime, meaning it is never hardcoded into the source code and cannot be accidentally exposed if the project is shared publicly on GitHub.

Because the Gemini free tier can occasionally return errors during periods of high demand, the bot includes a retry system. If a call to Gemini fails, the bot automatically retries up to three times with a three-second gap between attempts. If all three attempts fail, the bot falls back to a probability-based decision engine that uses pot odds and the Monte Carlo win estimate to make a sensible decision, ensuring the game always continues smoothly regardless of API availability.

## 3. Technical Design

---

### 3.1 Project Structure

The project is split into three separate Python packages, each with a clear responsibility:

- poker/ — all core game logic including cards, deck, hand evaluation, players, bots, game state and table orchestration. No GUI code exists in this package.
- probability/ — the Monte Carlo equity calculator, entirely independent of both the game engine and the GUI.
- gui/ — the tkinter interface, which calls into poker/ and probability/ but is never imported by them.

This separation means all game logic and probability calculations can be tested independently without any GUI, and the interface is simply a visual layer placed on top of already-verified logic.

### 3.2 Hand Evaluation

Rather than implementing poker hand ranking logic from scratch, the project uses the treys library, a tested, fast Python hand evaluator. Treys takes a player's hole cards, the community cards and

returns a numerical score where lower numbers represent stronger hands. This score is used both for determining the winner at showdown and to calculate the probability of having a winning hand

### 3.3 GUI Architecture

The GUI is built in tkinter using a frame-based layout. The window is divided into a main table area on the left and a fixed-width probability panel on the right. Bot turns are scheduled using `root.after()` — tkinter's built-in timer — with an 800ms delay between each bot action, giving the game a natural pacing without blocking the main GUI thread. As this is my first time using tkinter, I used the LLM Claude to help me debug the code.

## 4. Strengths of the Design

---

- Clean separation of game modules — the game engine, probability calculator and GUI are fully independent packages, making the codebase easy to navigate, test and extend.
- Probability calculator scales naturally — the same equity calculator works identically preflop, on the flop, turn and river simply by passing in the known board cards. No separate logic is needed per street.
- Resilient API integration — the retry and fallback system means the game is never broken by a temporary API outage, which is important for any production-facing integration.
- Secure key management — API credentials are never hardcoded, always loaded from environment variables, and excluded from version control.
- Genuinely distinct bot personalities — because Gemini interprets the personality prompt in context, bot behaviour varies in ways that a simple rule-based system could not replicate.

## 5. Areas for Future Development

---

- Blind structure - the current implementation does not post small and big blinds at the start of each hand. Adding this would make the game fully rules-compliant with standard Texas Hold'em.
- Card graphics - cards are currently displayed as text symbols such as A♠. Replacing these with actual card images would significantly improve the visual quality of the interface.
- Hand history log - a scrollable panel showing a record of previous hands and outcomes would help the player track how the session is progressing over time and aid in making the AI bots even more life-like
- Bot card reveal at showdown - currently, bot hole cards remain hidden throughout. Revealing them at showdown would make the result clearer and more satisfying to watch.
- Position awareness - in real poker, acting last is a significant strategic advantage. The bots currently have no awareness of their position relative to the dealer button.
- Multi-session statistics - tracking win rate, average pot size and total hands played across multiple sessions would add even more valuable insight for a user